

Claude Code Instructions — ATNX Mobile PWA + Server-Side Claude

We're executing a 4-step mobile enablement plan. Each step is a self-contained task. Do not proceed to the next step until the current one is verified working.

Context to load first: Read `atnx-web/README.md`, `atnx-extension/background.js`, `atnx-web/app/api/captures/route.ts`, and `atnx-web/lib/store.ts` before starting. The architecture is documented well — use it.

Step 1 — Unified server-side `/api/captures`

Goal: Eliminate the user-supplied Anthropic API key. The server does Claude vision analysis AND market creation in a single request. Breaking change is fine — only 3 users.

Changes:

- `/api/captures` POST now accepts `multipart/form-data: image` (File), `sourceUrl` (string, optional), `pageTitle` (string, optional), `pageContext` (string, optional — selected text or OG description).
- Server pipeline:
 1. Auth check via Supabase cookie (existing). 401 if no session.
 2. Upload image to `screenshots` bucket, get public URL.
 3. Call Claude vision with the image + context → produce the same analysis payload shape the extension currently sends (name, category, metrics_detected, platforms_detected, virality_signals, confidence_score, etc.). Keep the prompt identical to whatever the extension uses today so `composeVi` keeps working.
 4. Run existing `resolveOrCreateMarket` + `composeVi` + `recordVi` flow. No changes to `signals.ts`, `trends.ts`, `wikipedia.ts`, `llm-baseline.ts`.
 5. Return `{ marketId, isNew, vi, source }`.
- Add `ANTHROPIC_API_KEY` to server env. Never expose it to the client. Update `.env.local.example`.
- Do NOT add rate limiting. User asked to skip it for now.

The old JSON endpoint is gone. Don't keep a parallel path.

Verify: `curl` the endpoint with a test image + auth cookie, confirm a market is created and VI recorded.

Step 2 — Extension refactor

Goal: Extension no longer holds or uses an Anthropic API key. It sends the raw cropped image to the server.

Changes:

- `popup.html` / `popup.js`: remove the API key input entirely. Keep the Web App URL input.
- `background.js`: delete the Claude fetch call and all key-handling logic. After the offscreen doc returns the cropped image blob, POST it as `multipart/form-data` to `${webAppUrl}/api/captures` with `credentials: 'include'`. Include `sourceUrl` (current tab URL) and `pageTitle`.
- `chrome.storage.local`: remove `apiKey` reads/writes. Leave `webAppUrl`, `captureStatus`, `captureCount`.
- Error copy: keep the `401 → "Sign in at atnx.app first"` pattern, add a clear error for 500s from analysis failures.

Verify: Load the unpacked extension, sign into `atnx.app`, `Ctrl+Shift+X`, capture a meme — market appears on dashboard. No key anywhere in the extension.

Step 3 — PWA shell (manifest, SW, icons, install prompt)

Goal: `atnx.app` is installable on Android. Scroll-gated install prompt.

Changes:

- `atnx-web/public/manifest.json`:
 - `name`: "ATNX – Attention Exchange", `short_name`: "ATNX"
 - `start_url`: `"/app"`, `scope`: `"/"`, `display`: `"standalone"`
 - `background_color`: `"#000000"`, `theme_color`: `"#000000"` (CMYK identity — black canvas)
 - Icons: 192, 512, and a `"purpose": "maskable"` 512. Use the existing repo icon. If at 48px or inside the maskable safe zone the ATNX logo gets muddy (likely — small CMYK marks usually do), generate a simplified variant with just the logo mark on solid black, no wordmark. Flag this to me with both versions if you're unsure.
- Link the manifest from `app/layout.tsx` (`<link rel="manifest" href="/manifest.json" />`).
- Minimal iOS meta tags in `layout.tsx`: `apple-touch-icon`, `apple-mobile-web-app-capable`, `apple-mobile-web-app-status-bar-style = black-translucent`. Don't invest further in iOS — user deprioritized it.

- Service worker at `atnx-web/public/sw.js`: minimal — cache the app shell (`/app`, core JS/CSS), network-first for API routes, never cache `/api/captures` or `/api/markets/refresh`. Register from `layout.tsx` via a small client component. Do NOT use Workbox unless you already have a reason to — keep this hand-rolled and under 100 lines.
- Install prompt: client component that listens for `beforeinstallprompt`, stashes the event, and shows a custom banner only after the user has either (a) scrolled past ~600px on `/app`, or (b) viewed a market detail page. Suppress forever after dismiss (localStorage flag).

Verify: Chrome DevTools → Application → Manifest shows no errors. Lighthouse PWA audit passes installability. On a real Android device, visit `atnx.app`, scroll the dashboard, confirm the install banner fires, install, confirm it opens standalone.

Step 4 — Android share target + `/share` receiver

Goal: After install, ATNX appears in Android's "Share image" sheet. Sharing an image lands on `/share`, which auto-creates a market and redirects to the detail page — mirroring the extension's no-confirm UX.

Changes:

- Add to `manifest.json`:

json

```
"share_target": {
  "action": "/share",
  "method": "POST",
  "enctype": "multipart/form-data",
  "params": {
    "title": "title",
    "text": "text",
    "url": "url",
    "files": [{ "name": "image", "accept": ["image/png", "image/jpeg", "image/webp"] }]
  }
}
```

- `app/share/route.ts` (or page with a Server Action — your call, pick whichever is cleaner for handling the POST): parse the multipart payload, forward the image + `text/url` as `sourceUrl/pageContext` to `/api/captures` server-side (reuse the logic, don't re-implement it — probably extract the core pipeline from step 1 into a `processCapture()` function in `lib/store.ts` and call it from both routes).
- If no auth cookie → redirect to `/app` with a toast param saying "Sign in to create markets from shared images."
- On success → `redirect(/app/markets/${marketId})`.

- On failure → redirect to `/app` with error toast.
- Show a brief loading state during processing. Share targets are expected to respond fast; the Claude call takes a few seconds, so render a server component with an animated loader that the redirect replaces.

Verify: On installed Android PWA, share an image from Twitter/gallery/screenshot → ATNX appears in share sheet → tap it → lands on market detail with the new market created.

Working rules

- **Stop between steps.** After each step, report what changed, verification output, and wait for approval before the next.
- **Don't touch `signals.ts`, `trends.ts`, `wikipedia.ts`, `llm-baseline.ts`.** The VI pipeline is working.
- **Don't add rate limiting, mobile wallet adapter, iOS share shortcuts, protocol handlers, or anything else not in these 4 steps.** User scoped them out.
- **If something in the README is stale vs. the actual code, trust the code and flag the discrepancy.**
- **CMYK brand:** theme is black background, cyan/magenta/yellow accents. Don't introduce new colors in the install banner or `/share` loader — pull from existing Tailwind tokens.
- **Commit per step** with a clear message: `step 1: server-side claude in /api/captures`, etc.

Start with Step 1. Load the context files first, then propose the `/api/captures` handler diff before writing it.